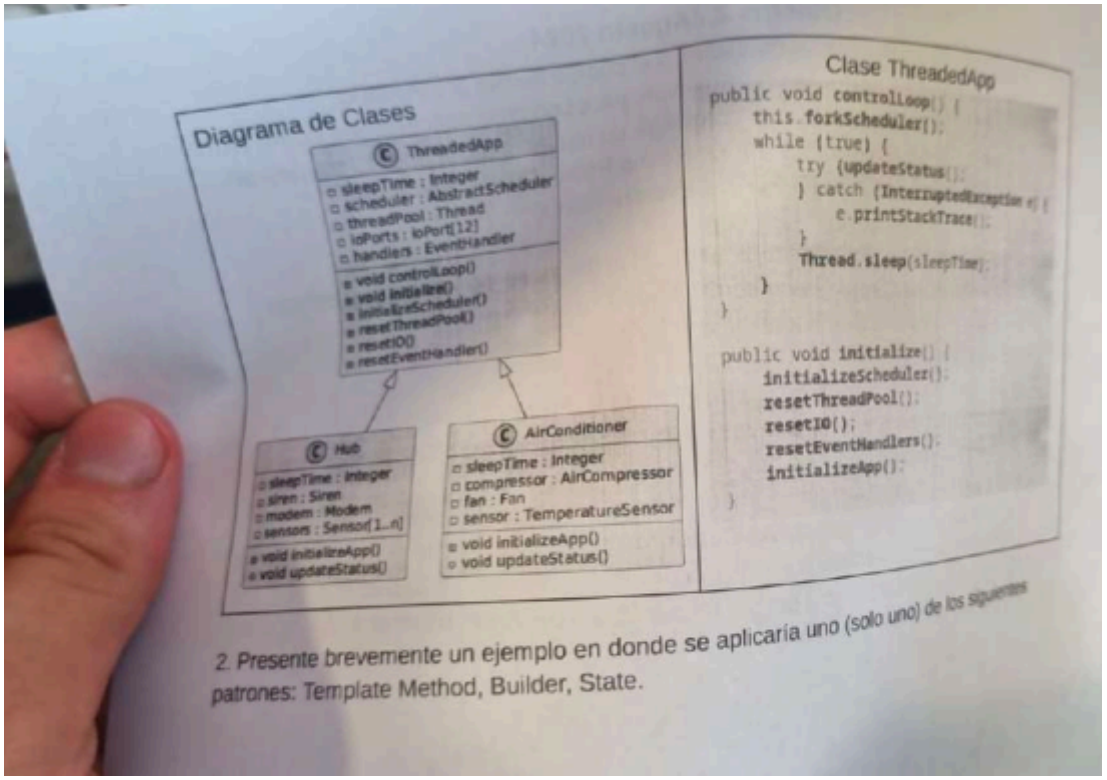
Final Agosto 2024

Objetos 2 - Agosto 2024

1. Considere el siguiente diagrama de clases y el código fuente presentado.
La clase ThreadedApp es parte de un framework para construir aplicaciones como Hub y AirConditioner, que es un sistema de control de un aire acondicionado. El framework incluye una jerarquía de Schedulers (FIFO, Round Robin, Priority, etc.), una abstracción de Threads, manejo IO Ports y EventHandlers.

Responda las siguientes preguntas considerando el diagrama y el código presentado

1. Explique brevemente que hace controlLoop()
2. Explique brevemente que hace initialize()
3. Liste los hotspots
4. Puede encontrar Inversión de Control? en qué clases y métodos?
5. Cómo se instancia el framework?
6. Este es un framework de caja negra o caja blanca?
7. La clase **AirCompressor** no se encuentra disponible pero se requiere implementar test de unidad de **AirConditioner** verificando que el los métodos de AirCompressor son enviados en las secuencias esperadas. ¿Explique si se pueden implementar los test unidad requeridos o se deber esperar a que la clase AirCompressor este disponible?



2. Presente brevemente un ejemplo en donde se aplicaría uno (solo uno) de los siguientes patrones: Template Method, Builder, State.

1. Explique brevemente qué hace control loop.

El método controlLoop() define el bucle principal de ejecución del framework,. Inicialmente, llama a this.forkScheduler().

Luego, entra en un ciclo infinito (while (true)) donde intenta ejecutar el método updateStatus(),. Si ocurre una excepción de interrupción, imprime la traza de la pila. Finalmente, pausa el hilo por el tiempo definido en sleepTime antes de repetir el ciclo,.
El método updateStatus()es el punto de variabilidad donde el framework cede el control a la aplicación. Está implementado en las clases Hub y AirConditioner para que cada aplización defina su lógica específica, por lo cual updateStatus() funciona como un método hook/gancho que define un hotspot dentro del esqueleto del algoritmo central del framework (control loop).
Este diseño sigue el patron template methos, donde la superclase ThreadedApp define la estructua de control, pero delega pasos variables a las subclases

2. Explique brevemente qué hace initialize()

1. Inicializa el planificador → initializeScheduler()
2. Reinicia el pool de hilos → resetThreadPool()
3. Reinicia configuraciones de I/O → resetIO()
4. Reinicia los manejadores de eventos → resetEventHandlers()
5. Llama al método de inicialización específico de la aplicación → initializeApp()

Al igual que en el caso anterior, el método initializeApp() es un metodo hook y es un punto de extensión dentro del framework.

3. Liste los hotspots

Los hotspots son los puntos de extensión que ofrece el framework para introducir variantes y contruir aplicaciones diferentes. Los hotspots del framework son:

- InitializeApp(): Llamado desde initialize(), permite a cada aplicacion que se construye con el framework, realizar su configuración específica
- updateStatus(): Llamado repetidamente desde controlLoop(), define la lógica que se va a ejecutar en cada ciclo del thread principal

4. Puede encontrar inversión de control? en qué clases y métodos?

La inversión de control es un mecanismo donde el framework toma el control de la ejecución e invoca el código específico dek desarrollador de la aplicación.

En el código proporcionado se puede encontrar inversión de control en los métodos:

- controlLoop() cuando se llama al método updateStatus()
- initialize() el cual llama al método initializeApp()

El código del framework ThreadedApp es quien envía mensajes al código de la aplicación (los métofdos initializeApp y updateStatus())

5. Cómo se instancia el framework?

El framework se instancia mediante subclasificación. El desarrollador crea una clase de aplicación concreta (Hub o AirConditioner) que hereda directamente de la clase principal del framework (ThreadedApp). Edta instanciación implica especializar y completar la funcionalidad incompleta del framework mediante la implementación de los métodos gancho.

6. Este es un framework de caja negra o de caja blanca?

Este es un framework de caja blanca ya que utiliza herencia como mecanismo de extensión. Esto requiere que el desarrollador de la aplicación (por ejemplo AirConditioner) conozca la estructura interna y el código del framework. La dependencia en la herencia es un indicador de que se trata de un framework de caja blanca ya que el loop de control se hereda directamente.

7. La clase AirCompressor no se encuentra disponible pero se requiere implementar test de unidad de AirConditioner verificando que el los métodos de AirCompressor son enviados en las secuencias esperadas. ¿Explique si se pueden implementar los test unidad requeridos o se deben esperar a que la clase AirCompressor este disponible?

Para lograr esto, se debe utilizar un Test Double que reemplace a AirCompressor (el Componente del que se Depende o sea DOC) Como el objetivo es verificar que los métodos de AirCompressor son llamados en las secuencias esperadas (lo que sería una verificacio de salidas indirectas), el Test Double más apropiado es un Mock Object,,.

El proceso sería:

1. Se crea y configura un Mock Object con las expectativas de las llamadas (métodos, argumentos y orden de secuencia) que AirConditioner (el SUT) debe enviar al AirCompressor,.
2. Se instala el Mock Object en el SUT.
3. Se ejercita el SUT, y el Mock Object verifica inmediatamente si las interacciones ocurren tal como se esperó,. Si la secuencia de llamadas no se cumple, el Mock Object provoca el fallo del test automáticamente,.

2. Presente brevemente un ejemplo en donde se aplicaría uno (solo uno) de los siguientes patrones: Template Method, Builder, State.

Patrón State

Modificar el comportamiento de un objeto cuando su estado interno se modifica, haciendo que externamente parezca que la clase del objeto cambió

Final Septiembre 2022

1. El equipo del banco ha tratado de implementar un patrón composite, lo que permite que traten a todos los tipos de cifradores (tanto los simples como el compuesto 'MultiCypher) de manera uniforme. Esto se logra haciendo que ambos tipos compartan una interfaz común (heredando de Cypher). Los elementos del patron composite que se pueden visualizar en la implementación proporcionada son:

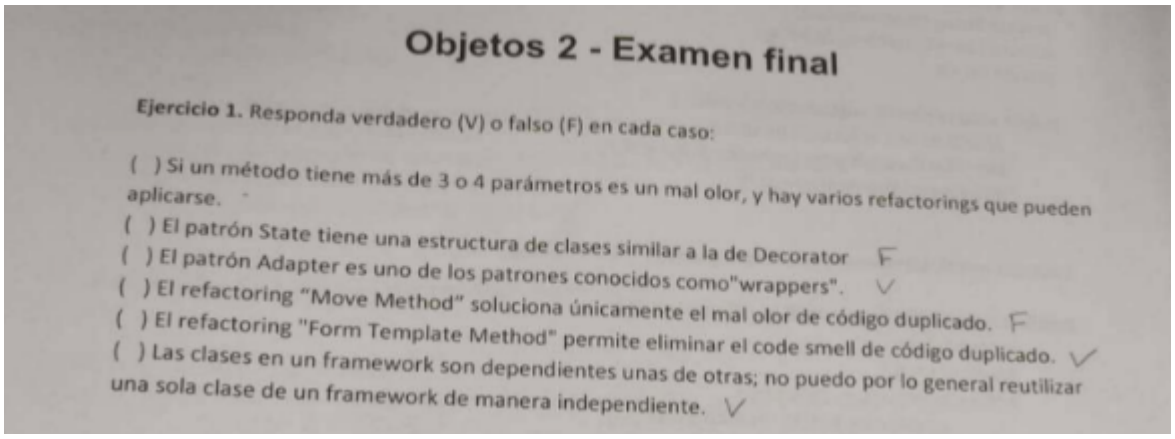
- Componente con una interfaz común: Se provee una clase abstracta Cypher que actua como interfaz Componente, definiendo operaciones comunes.
- Elementos primitivos y compuestos: Existen clases para los objetos individuales (RailFence, ColumnTransposition, RowTransposition) que actuan como Hojas (Leaf) y una clase que actua como componente compuesto.
- Estructura jerárquica recursiva: La clase MultiCypher representa un cifrador compuesto que internamente contiene otros objetos de tipo Cypher (que a su vez pueden ser otros MultiCypher), creando así una estructura de tipo árbol
- Operaciones para manipular a los hijos: La clase MultiCypher contiene operaciones específicas 'add' y 'remove' para manipular los componentes hijos (manejar los sub-árboles).

2. Es correcto que cipher y decipher sean abstractas ya que permite que todos los componentes, tanto las hojas como el composite, deban implementar estos métodos, lo cual es un uso inteligente del polimorfismo

Para mantener la uniformidad de la interfaz, La operación add debe declararse en el componente Cypher. Si bien al declarar este método como abstracto se obliga a las clases hoja a implementar este método aunque no lo necesiten, la idea de que add () sea un método abstracto es maximizar la transpoarencia y obligar explicitamente a cada subclase a tomar una decisión sobre cómo manejará la composición (obligar a las hojas a reconocer su incapacidad para almacenar componentes)

Si add() fuera un método concreto y vacío en Cypher (un *hook method* con implementación por defecto), la clase Compuesta (MultiCypher) lo heredaría sin implementarlo. Sin embargo, para que MultiCypher pudiera agregar funcionalidad real, tendría que sobrescribir dicho método. Aunque esto mantendría la interfaz limpia, la declaración *abstracta* obliga a la clase a tomar posición sobre su capacidad de composición. Además, el patrón Composite en su implementación más transparente suele requerir que las operaciones de gestión de hijos se definan en la interfaz base, aceptando la incomodidad de la implementación en las hojas para garantizar que el cliente nunca tenga que preguntar de qué tipo es el objeto antes de intentar llamarlo

FINAL SUELTO



Objetos 2 - Examen final

Ejercicio 1. Responda verdadero (V) o falso (F) en cada caso:

( ) Si un método tiene más de 3 o 4 parámetros es un mal olor, y hay varios refactorings que pueden aplicarse.

( ) El patrón State tiene una estructura de clases similar a la de Decorator F

( ) El patrón Adapter es uno de los patrones conocidos como"wrappers". V

( ) El refactoring "Move Method" soluciona únicamente el mal olor de código duplicado. F

( ) El refactoring "Form Template Method" permite eliminar el code smell de código duplicado. V

( ) Las clases en un framework son dependientes unas de otras; no puedo por lo general reutilizar una sola clase de un framework de manera independiente. V

1. Verdadero. Es el mal olor Long Parameter List
2. Verdadero. El adapter, proxy y decorator son los patrones conocidos como wrapper
3. Falso. También puede solucionar otros malos olores como por ejemplo Feature Envy.
4. Verdadero.
5. Verdadero. Las clases de una librería son independientes entre sí, pero las de un framework no.

Ejercicio 2. Suponga que debe refactorizar un sistema que permite aplicar distintas funciones o filtros sobre datos de entrada. El sistema está compuesto de módulos que se conectan entre sí de diferente forma para poder ir aplicando las funciones sobre los datos en una secuencia. Existe una jerarquía de distintos tipos de Módulos, cuya raíz es la clase Module (subclases SelectionModule, ArithmeticModule, SortingModule, etc.).

Suponga que tiene que realizar el refactoring con un compañero de trabajo. Ambos se encuentran con el código que está en la página siguiente y su compañero le dice:

a- "El único code smell que tiene este código es el uso de sentencias condicionales"

Totalmente de acuerdo Parcialmente de acuerdo ○En desacuerdo

b- "Este smell se resuelve aplicando el refactoring Extract Composite, que te lleva a tener el patrón Composite. Hacemos una jerarquía de conexiones, con una superclase Connection, una subclase SimpleConnection que cumpliría el rol de hoja, y una subclase CompositeConnection que a su vez tiene subclases ANDConnection, ORConnection y NEXTConnection."

○ Totalmente de acuerdo Parcialmente de acuerdo En desacuerdo

c- "Deberíamos mover el método enviarAlSiguiente() a la jerarquía de Conexion. Asi el módulo delegaría en su conexión el envío de los datos a el/los modulos siguientes"

○ Totalmente de acuerdo Parcialmente de acuerdo En desacuerdo

Si en alguna de las 3 expresiones de arriba marcó Parcialmente de acuerdo o En desacuerdo justifique.

```
public abstract class Module {
  private String tipoConeccion;
  private List<Modulo> colModSig;
  private int sig;

  public void recibirInput(List<Float> colD) {
        //colD es una colección de datos a procesar y enviar al/a los módulos conectados
        List<Float> result = this.procesarInput(colD);
        this.enviarAlSiguiente(result);
  }

  abstract public List<Float> procesarInput(List<Float> colD);

  public void enviarAlSiguiente(List<Float> colD) {
        int candidate;
        Random rand ;
        if (tipoConeccion == "simple")
                colModSig.get(1).recibirInput(colD);
        if (tipoConeccion == "AND")
                colModSig.forEach(mod -> mod.recibirInput(colD));
        if (tipoConeccion == "OR") {
                rand = new Random();
                candidate = rand.nextInt(colModSig.size()) ;
                colModSig.get(candidate).recibirInput(colD);
        }
        if (tipoConeccion == "NEXT") {
                (colModSig.get(sig).recibirInput(colD);
                if (sig < colModSig.size())
                        sig = sig +1;
                else
                        sig = 1;
        }
}}
```

Code Smells:
Método Largo
Condicionales

2 a. En desacuerdo. Tambien hay otros code smells como duplicated code,

primitive obsession, long method
2b. Totalmente de acuerdo (creo)
2c. No sé

Luego su compañero se ofrece a hacer el refactoring y vuelve al rato con el siguiente código que aparece en la próxima hoja, diciendo "solo falta que agregues la clase para el Next."
Elija una opción en cada inciso que sigue con respecto a la solución que escribió su compañero. Además justifique en los casos que no esté totalmente de acuerdo y corrija el código según su justificación. También agregue el código para la clase NextConnection.

d- Creo que la jerarquía que definió es correcta
Totalmente de acuerdo ○ Parcialmente de acuerdo En desacuerdo

e- Creo que aplicó el refactoring que indicó
Totalmente de acuerdo ○ Parcialmente de acuerdo En desacuerdo

f- Creo que las variables de instancia quedaron en el lugar correcto y no faltan ni sobran parámetros
Totalmente de acuerdo ○ Parcialmente de acuerdo ○ En desacuerdo

g- Creo que no quedan más refactorings por aplicar
Totalmente de acuerdo Parcialmente de acuerdo ○ En desacuerdo

```
public abstract class Module {
  private Connection con;
  private List<Module> colModSig;
  private int sig;

  public void recibirInput(List<Float> colD) {
        //colD es una colección de datos a procesar y enviar al/a los módulos conectados
        List<Float> result = this.procesarInput(colD);
        con.enviarAlSiguiente(colModSig, sig, result);
  }
                                  null
  abstract public List<Float> procesarInput(List<Float> colD);
}

public abstract class Connection {
  abstract public void enviarAlSiguiente(List<Module> colModSig, int sig, List<Float> colD);
}

public class SimpleConnection extends Connection {
  public void enviarAlSiguiente(List<Module> colModSig, int sig , List<Float> colD) {
        colModSig.get(1).recibirInput(colD);
  }
}

public abstract class CompositeConnection extends Connection {
}

public class ANDConnection extends CompositeConnection {
  public void enviarAlSiguiente(List<Module> colModSig, int sig , List<Float> colD) {
        colModSig.forEach(mod -> mod.recibirInput(colD));
  }
}

public class ORConnection extends CompositeConnection {
  public void enviarAlSiguiente(List<Module> colModSig, int sig , List<Float> colD) {
        int candidate;
        Random rand = new Random();
        candidate = rand.nextInt(colModSig.size()) ;
        colModSig.get(candidate).recibirInput(colD);
  }
}
```

CompositeConnection

FINAL SIN FECHA

1) (3p)
Identifique las semejanzas y diferencias entre los patrones State y Strategy en todos los aspectos que pueda (por ej., propósito, modelo de clases y modelo de instancias, cuestiones de implementación, etc.)?

-Ambos patrones permiten que un objeto cambie su comportamiento de forma dinámica y en tiempo de ejecución. Sin embarga el propósito de este cambio difiere en ambos patrones:

- El propósito del state es permitir que un objeto modifique su comportamiento cuando su estado interno cambia, haciendo que parezca que el objeto cambió de clase. Se enfoca en gestionar transiciones de estados de manera mas organizada.
- El propósito del strategy es definit una familia de algoritmos, encapsular cada uno y hacerlos intercambiables. Se enfoca en permitir que el algoritmo varie independientemente del cliente que lo utiliza

-Ambos patrones presentan un diagrama de clases similar ya que constan de una clase context que mantiente una referencia a una clase abstracta (State o Strategy) y una jerarquia de clases concreta que implementan dicha interfaz
-Ambos utilizan la composición de objetos y la delegación para evitar problemas de diseño como sentencias switch y obsesión por los primitivos
-Mientras que un Strategy suele tener solo un mensaje público (suele estar diseñada para resolver una única tarea), un State puede definir múltiples mensajes que corresponden a muchas operaciones del contexto que dependen del estado actual.
-Mientras que en el patrón State los ConcreteState suelen conocerse entre si y son responsables de saber que estado le siue a otro y bajo que circunstancias realizar la transición, en el patrón Strategy las ConcreteStrategy son completamente independeientes entre si y no se conocen entre ellas
-En el patrón State el estado suele ser priavado del Contexto, mientras que en el Strategy las estrategias posible son conocidas por fuera para que el cliente pueda configurarla en el contexto.
-El patrón State genera mayor acoplamiento entre las subclases de la jerarquía ya que son estas quienes deben gestionar las transiciones de estados, mientras

que el patrón desacopla a lcontexto de los detalles de implementación de los algoritmos

2. Resuelva el siguiente ejercicio (7p)

Considere el código que se presenta a continuación, correspondiente a la clase **E-Shop** que representa un sitio de compras online.

```
E-Shop>>upgradeAccount: cu
    "Recibe una instancia de Customer como parámetro. Actualiza la cuenta de cu por una mejor"

    cu account type = 'Bronce'
      ifTrue: [
            cu account: (Account new type: 'Plata').
            cu resetPoints.
            ^cu history add: (HistoryEvent new text: 'Cuenta actualizada a Plata'
                                                    on: Date today) ].
    cu account type = 'Plata'
      ifTrue: [ | s |
            s := cu account transactions inject: 0 into: [: t | t amount].
            s < 1000
              ifTrue: [
                    cu account: (Account new type: 'Oro').
                    cu points: (cu points * 1.10).
                    ^cu history add: (HistoryEvent new text: 'Cuenta actualizada a Oro'
                                                    on: Date today) ]
              ifFalse: [ "s > 1000"
                    cu addToShopCart: (self pdfForBook: 'Enterprise Scrum' value: 0).
                    cu account: (Account new type: 'Platino').
                    cu points: ( cu points * 1.25).
                    ^cu history add: (HistoryEvent new text: 'Cuenta actualizada a Platino'
                                                    on: Date today)]]
```

Considere que a futuro se planean tener nuevos tipos de cuenta.

envidia de atributos
SWITCH
Long Method
nombre fea

2.1) Enumere todos los malos olores que puede identificar en el código.
2.2) Por cada uno de los problemas identificados, describa cómo lo solucionaria en términos de refactoring. Si utiliza algún patrón en su solución justifique su elección.
2.3) Dibuje el diagrama de clases de su solución final y escriba el código resultante después de aplicar todos los refactorings de 2.2.

Code Smells:

Feature Envy: Aparce cuando una clase se ve mas interesada en datos y métodos de otra clase que en los propios (se muestra "envidiosa" de las capacidades de otra clase). Esto es indicativo de un problema de diseño (que el método o parte del método esta mal ubicado)

a. Extract Method
    i. Se utiliza extract method para aislar las partes del código que presentan envidia de atributos en metodos propios → en este caso vemos que todo el método presenta envidia

b. Move Method
    ii. Se mueve le metodo upgradeAccount a la clase Costumer

Duplicated code: El mismo código o código similar aparece en varias partes del código. Vemos que la logica para añadir un envento está duplicada.

a. Extract Method: se crea un metodo para manejar el agregado de un evento en un método reutilizable

Long Method: El método upgradeAccount hace muchas cosas (tiene muchas responsabilidades)y tiene muchas lineas de código

a. Extract Method: Al igual que en el caso anterior, al reemplazar lógica por llamados a métodos en la clase customer se soluciona este code smell

Primitive Obsession: Obsesión por tipos primitivos en lugar de objetos con significado propio, lo cual refuerza estructuras rígidas y dificulta validaciones. Como se ve que cada tipo de cuenta tiene un comportamiento asociado, esto nos indica que debería ser una entidad conceptual, no un tipo primitivo

a. Replace State Altering Constat with State: Este refactoring consiste en crear una jerarquia en donde cada subclase represente un estado concreto en el que se puede encontrar el contexto para que dichas subclases manejen las transiciones entre los posibles estados
    i. Aplico “Replace type code with class” para crear una clase que será la superclase del State
    ii. Aplico “Extract subclass” para crear una subclase del state por cada estado concreto en el que se puede encontrar el contexto
    iii. Para método del contexto que contenga condicionales que chequean por el estado
        1. Aplico “Move Method” a la superclase del State
        2. Paso el contexto como parametro si es necesario
    iv. Para estado concreto/subclase
        1. aplico un push down method para mover desde la superclase os métodos que tienen lógica de transición de estados
        2. elimino comprobaciones de estrado
    v. Declaro como abstracto el método de la superclase. .

Switch Statements: Hay secciones condicionales que contiene lógica para

distintas tipos de objetos que son instancias de la misma clase

a. Replace Conditional with Polymorphism
    i. Crear una jerarquia de clases donde cada subclase representa un caso del condicional.
    ii. Si el condicional es parte de un método mas largo entonces aplicar extract method para aislar el comportamiento variable
    iii. Por cada subclase:
        1. crear un método que sobreescriba el método que contiene el condicional de la superclase
        2. Copiar la sección del condicional correspondiente
        3. eliminar del método de la superclase la branch que acabo de copiar
    iv. hacer el método de la superclase abstracto.
    v. copular y testear.

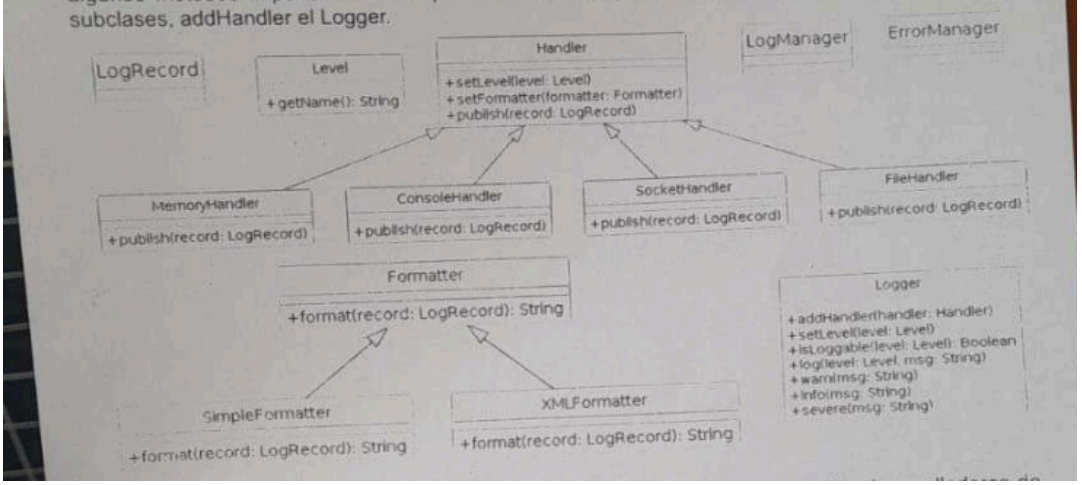
UML:

Parcial Frameworks 1er Recuperatorio 2024

Ejercicio 3 - Frameworks

OO2 -1er recuperatorio- 29/06/2024

Consideremos el framework de Logging visto en la materia. El framework permite loguear eventos importantes, errores, etc. Estos logs son útiles para desarrolladores, administradores y usuarios. El framework permite enviar mensajes de log a consola, archivos, sockets, etc.
Como ayuda memoria se ofrece un diagrama simplificado de algunas clases del framework. Note algunos métodos importantes como publish en handler y sus subclases, format en formatter y sus subclases, addHandler el Logger.



El framework se encarga de crear y gestionar las instancias de Loggers que los desarrolladores de una aplicación utilizarán para loguear. La configuración de los loggers generalmente se realiza al iniciar la aplicación (y se mantiene globalmente). Esto significa que se pueden establecer niveles de log, formatos de salida, destinos de los logs y otras opciones relacionadas con la generación y gestión de logs.

1. Se está desarrollando una aplicación que requiere usar este framework de la siguiente manera: Se requiere tener dos loggers, uno para loguear en consola (salida a pantalla) y otro para loguear en un archivo con formato XML. Explique brevemente cómo utilizaría el framework para implementar esta funcionalidad (cómo lo configuraría, que clases provistas utilizaría, qué clases se deben extender, qué métodos se deben agregan o implementar, etc).

2. En otra aplicación se requiere utilizar el framework para utilizar un logger que preste atención a los eventos a partir del nivel SEVERE y enviarlos por WhatsApp. Explique brevemente cómo utilizaría el framework para implementar esta funcionalidad (cómo lo configuraría, que clases provistas utilizaría, qué clases se deben extender, qué métodos se deben agregan o implementar, etc). Para enviar un mensaje por whatsapp escriba la siguiente expresión: Whatsapp.enviar(String mensaje, String teléfono).

3. Se necesita desarrollar una aplicación utilizando este framework para publicar los logs por mail. ¿Puede asegurar que en la implementación de esta funcionalidad el programador notará la existencia de inversión de control? Justifique su respuesta en términos de que debe hacer el programador y qué observa cuando el código se ejecuta.

1. Cada aplicación cuenta con un logger que posee un nivel que compra con el nivel de mensaje que recibe para aceptarlo si es igual o mas relevante que el suyo o rechazarlo si es menor. A su vez se cuenta con un handler que es el encargado de enviar el mensaje a donde corresponda. Este cuenta a su vez con un nivel y con un formatter que determina cómo se presentará el mensaje.

Las instancias del handler son consolehandler, memoryhandler, filehandler y sockettoragehandler. Por defecto el consolehandler tiene el simpleformatter y el filehandler tiene xmlformatter.
Para usar un logger instancio un logger y le pido el logger con el nombre que le pase por parametro, si no existe lo crea.Entonces para hacer un logger que

loguee en consola:

//creo una instancia de logger
Logger logger=Logger.getLogger("app")
//lo seteo con un nivel que quiera logger.setLevel(Level.SEVERE)
como por defecto el handler es consolehandler, al llamar al logger y pasarle el mensaje, lo imprime en pantalla

logger.log(Level.SEVERE,"HOLA")

Para loguear en un archivo con formato xml le paso al handler un nuevo formatter que va a ser el xml

Logger loggerXML=Logger.getLogger("app.xml") FileHandler file=new FileHandler("archivo.xml") file.setLevel(Level.SEVERE) logger.addHandler(file)

como filehandler tiene por defecto el xmlformatter solo instancio el logger con el mensaje que quiero que el handler envie al archivo xml

loggerXML.log(Level.SEVERE, "holu")

2. Necesito crear una clase que extienda de Handler (WhatsappHandler) la cual se encargará de manejar el envio de mensajes (logs).Ademas necesito sobrescribir el método publis(LogRecord record) ya que necesito invocar al metodo de la clase Whatsapp que envia el mensaje:

```java
public class WhatsAppHandler extends Handler {
    private String telefono;

    public WhatsAppHandler(String tel) {
        this.telefono = tel;
    }

@Override
public void publish(LogRecord record) {
        if (record.getLevel().equals(this.getLevel())) {
            Whatsapp.enviar(record.getMessage(), telefono);
        }
    }

@Override public void flush() {}
@Override public void close() throws SecurityException {}
 }
```

w

```java
Logger logger = Logger.getLogger("MiLoggerSevere");
 logger.setLevel(Level.SEVERE);
wspHandler= new
 WhatsAppHandler("+541112345678")
 wspHandler.setLevel(Level.SEVERE)
 logger.addHandler(wspHandler);
```

3. Sí, el programador nota la inversión de control porque no llama directamente a los métodos que hacen el logging.
El programador solo llama logger.log(...) etc
No sabe qué Handlers se ejecutarán, en qué orden, ni qué formato tendrán.
El framework controla qué hacer, cómo, y cuándo hacerlo.A que métodos definidos por el


```java
Logger logger = Logger.getLogger("MiLoggerSevere");
 logger.setLevel(Level.SEVERE);
wspHandler= new
 WhatsAppHandler("+541112345678")
 wspHandler.setLevel(Level.SEVERE)
 logger.addHandler(wspHandler);
```

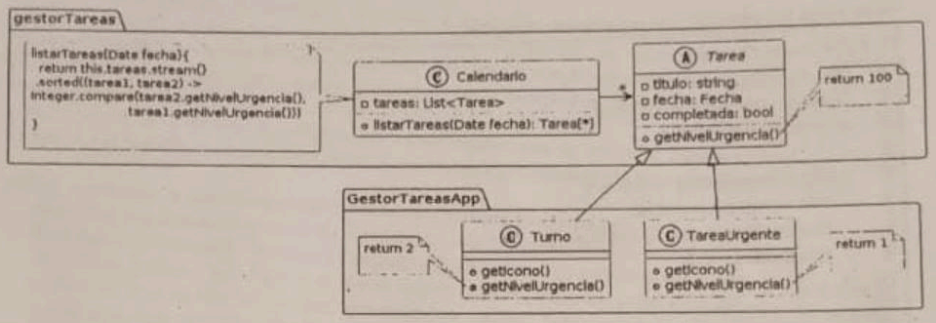
programador que está usando el framework llamar (esto es inversión de control).
El flujo lo maneja el framework, el programador solo configura o extiende.

Hay inversión de control porque el flujo de ejecución del logging no lo controla el programador, sino el framework. El programador configura Handlers y Formatters, pero no llama directamente a sus métodos, el framework se encarga.

Parcial 1ra fecha 2024

Ejercicio Frameworks — OO2 - Parcial 8/6/2024

Consideremos una parte de un framework muy simple para la gestión de tareas. El framework define un calendario que permite registrar muchas tareas. Al utilizar este calendario, es necesario especificar qué tareas se van a incluir. Además, ofrece la posibilidad de listar las tareas para una fecha determinada en orden de prioridad, comenzando desde las menos urgentes hasta las más prioritarias.



Para crear una aplicación con el framework, el programador debe:

- Crear una instancia de Calendario y agregarle tareas.
- Las tareas deben ser subclases de Tarea, y puede definirse libremente según se requiera.
- Al definir una tarea tiene que determinar cuál será el nivel de urgencia de la misma. Se espera que las tareas retornen un nivel de urgencia que debe ser un número entero. Cada tarea puede definir un nivel de urgencia para registrar su prioridad, que se utiliza entre otras cosas, para el listado.
- Este framework ordena las tareas para un día del calendario por nivel de urgencia, considerando las de mayor valor como las más urgentes.

GestorTareasApp es una aplicación construida con el framework. En ella los programadores definieron dos tipos de tareas: Turno y TareaUrgente, ambas subclases de Tarea. La tarea urgente tiene un nivel de urgencia 1, mientras que Turno tiene un nivel de urgencia 2.

Tareas:

1) Responda las siguientes preguntas, basándose en el subconjunto de clases y métodos del framework presentado anteriormente:
   a) A los programadores de la aplicación GestorTareasApp, no les gusta el orden ofrecido por nivel de urgencia, considerando las de mayor valor como las más urgentes. ¿Pueden modificar ese orden? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?
   b) Los programadores requieren nuevas tareas para su aplicación. ¿Pueden agregarlas? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?
   c) Los programadores de la aplicación midieron los tiempos de respuestas al momento de ordenar tareas y les parece muy lento. Quieren usar una colección que ordene de manera eficiente. ¿Pueden hacerlo? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?
   d) ¿Se puede implementar una tarea que implemente el nivel de urgencia incrementando su valor al acercarse la fecha límite? En base a su respuesta, ¿diría que es un frozen spot o un hot spot?

2) ¿Observa métodos gancho? ¿Cuáles?
3) Analizando la clase Turno, los programadores de GestorDeTareasApp pueden asegurar si hay una inversión de control? ¿Dónde?

1- a:
No se puede modificar el orden ofrecido por listarTareas, ya que este método se encuentra definido dentro del framework en la clase gestorTareas y no ofrece mecanismos explícitos de extensión ni de sobreescritura para cambiar la política de ordenamiento (que está basada en el método getNivelUrgencia). Por lo tanto, este comportamiento corresponde a un frozen spot.
1-b:
Sí, los programadores pueden agregar nuevas tareas creando nuevas subclases de Tarea. El mecanismo de extensión del framework es la herencia, permitiendo redefinir el método getNivelUrgencia() según la lógica propia de cada tipo de tarea. Por lo tanto, este punto corresponde a un hot spot, ya que permite a los desarrolladores extender el comportamiento del framework.
1-c:
No, los programadores no pueden cambiar la colección utilizada para almacenar las tareas, ya que el framework fija internamente una List<Tarea> en la clase Calendario, sin ofrecer mecanismos de extensión ni reemplazo de esta estructura. Por lo tanto, este aspecto corresponde a un frozen spot.
1-d:
Sí, se puede implementar una tarea que modifique dinámicamente su nivel de urgencia en función de la cercanía a la fecha límite, implementando la lógica deseada en el método getNivelUrgencia() de una subclase de Tarea. Este es un hot spot, ya que el framework permite y espera que los desarrolladores redefinan este método para personalizar el comportamiento.
2:
Sí, se observa un método gancho. El método plantilla listarTareas del framework ordena las tareas utilizando getNivelUrgencia(), el cual es un método gancho que las subclases de Tarea pueden redefinir para controlar el nivel de urgencia.

3:
Sí, hay inversión de control. Los programadores definen subclases de Tarea con sus propios métodos, pero no controlan directamente cuándo ni cómo se invocan estos métodos. Es el framework, a través del método listarTareas, el que controla el flujo de ejecución, invocando getNivelUrgencia() de cada tarea según lo necesite.

Ejercicios de parciales

Parcial 2025 1ra fecha

Refactoring

code smells:

reinventando la rueda: usar las herramientas que proporciona el lenguaje, que ya están testeadas y sabemos que funcionan

- lineas 10 a 12
- lineas 29 a 31
-

switch statements: este smell aparece cuando tenemos lógica condicional para distintos tipos de objetos. Esto genera que la misma estructura condicional (sentencias if-else/switch) aparezcan repetidas en varios lugares.
En el código proporcionado, este chequeo se realiza para verificar el tipo del documento

- lineas 6 y 25

duplicated code: sucede cuando el mismo código o codigo muy similar aparece en varios lugares. Esto agranda innecesariamente el tamaño del código y dificulta su mantenimiento

- lineas de 8 - 13 son iguales a lineas 27-32
- lineas 14-16 son muy parecidas a lineas 33-35

long method: El método generateReport es demasiado largo (mas de 20 lineas) loi que dificulta el entendimiento, mantenimiento y uso del mismo.

- lineas 6-43

primitive obsession: este smell implica el uso excesivo de tipos primitivos en lugar de objetos con significado. Esto disminuye la expresividad del código y refuerza estructuras rígidas. En el código se puede observar que se utiliza una variable de tipo 'String' para almacenar el tipo del documento, en vez de utilizar un objeto con significado propio

- linea 2

Refactoring:
Despues de cada refactoring aplicado se compila y testea el código
Paso 1: resolver smell Reinventando con el refactoring s*ubstitute Algorithm*

Este refactoring consiste en reemplazar un algoritmo complicado por uno más claro y directo, aprovechando las herramientas del lenguaje,. En lugar del for manual, deberías usar un método del lenguaje (como String.join o un recolector de streams) que ya esté testeado y garantice un funcionamiento óptimo,.

antes

Paso 2

pasos detallados. Como es un refactoring to patterns tiene varios refactoring juntos porque combina muchos refactorings para llevar el código a una estructura de un patrón, en este caso el strategy.

1. Creamos una clase estrategia
2. hacemos un 'move method' donde aparece el switch statement para que la clase estrategia se encargue de la misma
    a. Definir una variable de instancia para que haga referencia a la estrategia junto a su setter en la clase contexto.
    b. Dejar un método en el contexto que delegue el comportamiento referenciando y llamando a la estrategia.
    c. Elegir los parámetros necesarios para pasar al strategy
    d. compilar y testear
3. Aplicar extract parameter en el código del contexto que inicializa un strategy concreto para permitir a los clientes setear el strategy (pasarlo como parámetro en el constructor)
4. Aplicar replace conditional with polymorphism en el método strategy lo que este refactoring es reemplazar el switch statements separando cada lógica en una clase distinta aplicando asi mismo polimorfismo
5. compilar y testear con distintas combinaciones de estrategias y contextos.

Paso 3: aplicar from template method para solucionar el mal olor duplicated code

pasos detallados.

3. Reemplazar Condicional con Polimorfismo (*Replace Conditional with Polymorphism*)

• Problema: Las sentencias if-else (líneas 6 y 25) verifican el tipo de objeto para decidir qué lógica ejecutar,.
• Aplicación: Mueves cada rama del condicional a un método sobrescrito en una subclase de la jerarquía creada en el paso anterior (ej. PdfStrategy, XlsStrategy),.
• Resultado: El método generateReport original dejará de tener if-else y simplemente delegará la ejecución al objeto estrategia actual,. Esto permite añadir nuevos formatos (como CSV) simplemente creando una nueva subclase, cumpliendo con el principio de abierto/cerrado,.
4. Formar el Método Plantilla (*Form Template Method*)
• Problema: Existe código duplicado significativo: la creación de docFile y el seteo de metadatos (líneas 8-13 y 27-32) son casi idénticos en ambos formatos,.
• Aplicación: Una vez que tienes las subclases por formato, notarás que realizan pasos similares en el mismo orden,. La mecánica consiste en:

1. Descomponer los métodos de las subclases para identificar qué partes son idénticas y cuáles son únicas,.
2. Subir las partes idénticas (como la configuración inicial de metadatos) a la superclase,.
3. Estandarizar las firmas de los pasos únicos (ej. el método que llama al exporter específico) para que tengan el mismo nombre en todas las subclases,.
4. Elevar el método esqueleto a la superclase, el cual invocará los pasos comunes y llamará a los métodos abstractos o ganchos (*hooks*) que cada subclase implementará de forma específica,,.

Parcial 2024 2da Fecha:

Refactoring

Feature Envy
Data Class
Reinventando la rueda
Long method?
Reinventando la rueda
Primitive Obsession
Duplicated Code

Divergent change ??

Code Smell: **Feature Envy**

- Este code smell aparece cuando un método o parte de un método de una clase utiliza datos y métodos de otra clase para realizar su trabajo (se muestra envidiosa de las capacidades de otra clase). Esto es indicador de un problema de diseño (que el método o ciertas lineas del método están mal ubicadas)
- En el código la clase Pago calcula el precio con IVA de cada producto. Esta responsabilidad debería ser de la clase producto

Refactoring que lo corrige:

- **Extract Method**: Primero se extrae la parte del código que presenta envidia en un método propio
- **Move Method:** Luego se reubica ese método en la clase Producto

Código resultante después del refactoring

luego de aplicar el **extract method**:

```
public class Pago {
      private List <Producto> productos;
      private String tipo;
      //resto del código

      public double totalConIVA(Producto producto){
            return (producto.getPrecio()+ (producto.getPrecio() *
producto.getIVA()));

      }

      public double calcularMontoFinal(){
            double total= 0.0;
            if (this.tipo == "EFECTIVO"){
                  for (Producto producto: this.productos){
                        total= total+ this.totalConIVA(producto);
                  }
                  if (total >100000){
                        total= total -DESCUENTO_EFECTIVO;
                  }
            }
            else if (this.tipo == "TARJETA"){
                  for (Producto producto: this.productos){
```

```
                        total= total+ this.totalConIVA(producto);
                  }
                  total= total + ADICIONAL_TARJETA;
            }
            return total;
      }
}
```

luego de aplicar el **move method:**

Nota: Este refactorig también soluciona el code smell **Data Class** el cual aparece cuando una clase solo actua como contenedor de datos (como la clase Producto). Al aplicar este refactoring se le añade lógica propia y responsabilidades a la clase Producto, solucionando el mal olor.

```
public class Pago {
      private List <Producto> productos;
      private String tipo;
      //resto del código

      public double calcularMontoFinal(){
            double total= 0.0;
            if (this.tipo == "EFECTIVO"){
                  for (Producto producto: this.productos){
                        total= total+ producto.totalConIVA();
                  }
                  if (total >100000){
                        total= total -DESCUENTO_EFECTIVO;
                  }
            }
            else if (this.tipo == "TARJETA"){
                  for (Producto producto: this.productos){
                        total= total+ producto.totalConIVA();
                  }
                  total= total + ADICIONAL_TARJETA;
            }
            return total;
      }
}

public class Producto {
      private double precio;
      private double IVA;

      //resto del código
```

```
	public double totalConIVA(){
		return (this.getPrecio()+ (this.getPrecio() *
this.getIVA()));

	}
}
```

Code Smell: Reinventando la rueda

- Se utiliza una forma de calcular el total para los productos poco eficiente, siendo que ya existen estructuras brindadas por el lenguaje que estan testeadas y son seguras.

Refactoring que lo corrige: Substitute Algorithm

Código resultante después del refactoring:

```
public class Pago {
	private List <Producto> productos;
	private String tipo;
	//resto del código

	public double calcularMontoFinal(){
		double total= 0.0;
		if (this.tipo == "EFECTIVO"){
			total=
this.productos.stream().mapToDouble(p->p.totalConIVA).sum();
			if (total >100000){
				total= total -DESCUENTO_EFECTIVO;
			}
		}
		else if (this.tipo == "TARJETA"){
			total=
this.productos.stream().mapToDouble(p->p.totalConIVA).sum();
			total= total + ADICIONAL_TARJETA;
		}
		return total;
	}
}

public class Producto {
	private double precio;
	private double IVA;

	//resto del código
```

```
	public double totalConIVA(){
		return (this.getPrecio()+ (this.getPrecio() *
this.getIVA()));

	}
}
```

Code Smell: **Duplicated Code**

- Este code smell aparece cuando tenemos el mismo código o código muy similar que aparece en varios lugares del código.

Refactoring a utilizar:

- **Extract Method**: Extraemos el código duplicado en un método para reutilizarlo y asilar el comportamiento comun.

Código refactorizado:

```
public class Pago {
	private List <Producto> productos;
	private String tipo;
	//resto del código

	public double costoBase(){
		return
this.productos.stream().mapToDouble(p->p.totalConIVA).sum();

	}
	public double calcularMontoFinal(){
		double total= 0.0;
		if (this.tipo == "EFECTIVO"){
			total = this.costoBase();
			if (total >100000){
				total= total -DESCUENTO_EFECTIVO;
			}
		}
		else if (this.tipo == "TARJETA"){
			total= this.costoBase();
			total= total + ADICIONAL_TARJETA;
		}
		return total;
	}
}

public class Producto {
	private double precio;
```

```
	private double IVA;

	//resto del código

	public double totalConIVA(){
		return (this.getPrecio()+ (this.getPrecio() *
this.getIVA()));

	}
}
```

Code smell: Primitive Obsession

- Es el uso excesivo de tipos primitivos en lugar de objeto con significado, lo cual disminuye la expresividad del código y refuerza estructuras rígidas.
- En el código aparece al utilizar una variable de tipo String para representar el tipo de pago en lugar de objetos con significado propio

Refactoring a utilizar:

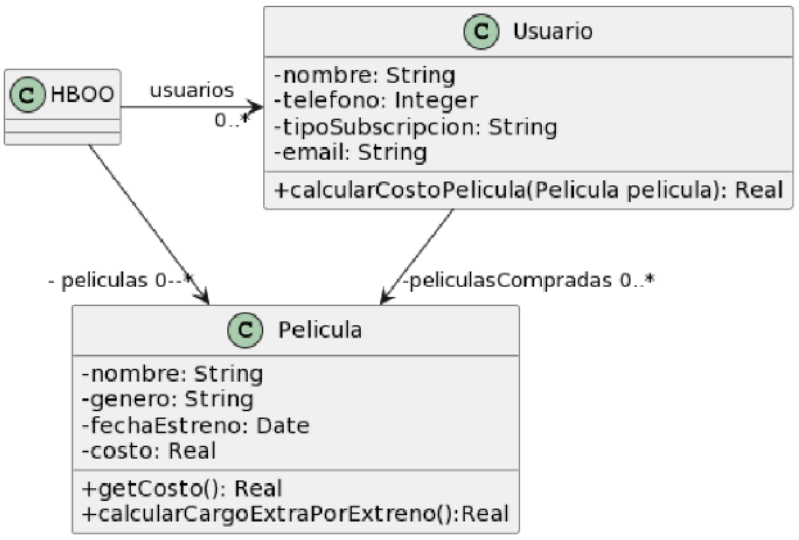
- Replace Conditional with Strategy: Los pasos para llevar a cabo este refactoring son:
    1. Crear la clase Strategy
    2. Hacer un Move Method de los condicionales de la clase contexto a la strategy (ajustando la firma de ser necesario)
        a. Crear una variable en el contexto que referencie al strategy
        b. Creamos un método setter para poder
    3. Relizar

Código refactorizado:

Ejercicios de práctica

Refactoring:

Ejercicio 2.6 - Películas:




```
public class Usuario {
    String tipoSubscripcion;
    // ...

    public void setTipoSubscripcion(String unTipo) {
     this.tipoSubscripcion = unTipo;
    }

    public double calcularCostoPelicula(Pelicula pelicula) {
     double costo = 0;
     if (tipoSubscripcion=="Basico") {
      costo = pelicula.getCosto() +
pelicula.calcularCargoExtraPorEstreno();
     }
     else if (tipoSubscripcion== "Familia") {
      costo = (pelicula.getCosto() +
pelicula.calcularCargoExtraPorEstreno()) *
0.90;
     }
     else if (tipoSubscripcion=="Plus") {
      costo = pelicula.getCosto();
     }
     else if (tipoSubscripcion=="Premium") {
      costo = pelicula.getCosto() * 0.75;
     }
     return costo;
    }
```

```
}

public class Pelicula {
    LocalDate fechaEstreno;
    // ...

    public double getCosto() {
     return this.costo;
    }

    public double calcularCargoExtraPorEstreno(){
// Si la Película se estrenó 30 días antes de la fecha actual, retorna
un cargo
de 0$, caso contrario, retorna un cargo extra de 300$
return (ChronoUnit.DAYS.between(this.fechaEstreno, LocalDate.now()) ) >
30 ? 0 :
300;
}
}
```

Code Smell:

- **Switch Statements**: El método 'calcularCostoPelícula' de la clase Usuario usa una estructura if-else-if para determinar un comportamiento (calcular el costo de la película) en base al valor de la variable 'tipoSubscripcion'. Este es un mal olor ya que genera código duplicado y dificulta el mantenimiento del mismo al no aprovechar el polimorfismo que brinda la POO (el polimorfismo debería ser el encargado de decidir qué cálculo aplicar en lugar de realizar comprobaciones explícitas)
  Como todos los objetos son instancias de la misma clase 'Usuario', la solución es crear subclases
-

Refactor:

- Replace

Code Smell:

- **Primitive Obsession**: En la clase Usuario se utiliza una variable de tipo String 'tipoSubscripcion' para representar un concepto de dominio que tiene reglas de negocio asociadas. Esto es un mal olor ya que disminuye la expresividad del código, dificulta

las validaciones y refuerza estructuras rígidas.
En lugar de modelar la suscripción del usuario como un string (un tipo primitivo) debería modelarse como una clase que tenga su propio comportamiento asociado.

Refactor:

- **Replace Type Code with Strategy// Replace Conditional Logic with Strategy**: Dado que el tipo de suscripción de un usuario es algo que podría cambiar en un futuro lo ideal sería delegar el comportamiento del cálculo a cada clase de tipo Suscripción, las cuales deberían compartir interfaz para hacerlas intercambiables.
  Para aplicar este refactoring to patterns se deben seguir los siguientes pasos:
  1. Crear clase Strategy (en este caso una interfaz ya que no existe comportamiento común entre los tipos de suscripción, solo necesitamos definir la firma del método)

```
public interface tipoSuscripción {
  public double calcularCosto( Peliucula pelicula);

}
```

  2. Crear un Strategy por variante:
  3. Aplicar move method para mover la lógica del contexto al strategy:

```
public class Basico implements tipoSuscripcion{
    public void  Basico(){
        super();
    }
    @Override
    public double calcularCosto(Pelicula pelicula ){
        return pelicula.getCosto() +
película.calcularCargoExtraPorEstreno();
     }

}
```

```
public class Familia implements tipoSuscripcion{
    public void  Familia (){
```

```
        super();
    }
    @Override
    public double calcularCosto(Pelicula pelicula ){
        return pelicula.getCosto() +
película.calcularCargoExtraPorEstreno()* 0.90;
     }

}
```

```
public class Plus implements tipoSuscripcion{
    public void  Plus (){
        super();
    }
    @Override
    public double calcularCosto(Pelicula pelicula ){
        return pelicula.getCosto();
     }

}
```

```
public class Premium implements tipoSuscripcion{
    public void  Premium (){
        super();
    }
    @Override
    public double calcularCosto(Pelicula pelicula ){
        return pelicula.getCosto()*0.75;
     }

}
```

4. Definir la variable de instancia en el contexto y agregar un setter para permitir que sean intercambiables:

```
public class Usuario{
    tipoSuscripcion suscripcion;
    // ....
    public void setSuscripcion (tipoSuscripcion suscripcion){
        this.suscripcion = suscripcion;
    }
    //...
}
```

5. Dejar un metodo en el contexto que delegue a la estrategia:
6. Elegir qué parámetros pasar:
7. Aplicar Replace conditional with polymorphism para eliminar las sentencias if-else-if:

```
public class Usuario {
  TipoSuscripcion suscripcion;
  public void setSuscripcion(TipoSuscripcion suscripcion) {
    this.suscripcion = unTipo;
  }

  public double calcularCostoPelicula(Pelicula pelicula) {
    return this.suscripcion(pelicula);
  }
}
```

8.

- Crear un **Strategy** por variante.
- Hacer que el método delegue en el strategy.

<u>Mecánica</u>

1. Crear clase Strategy.
2. Move Method: mover lógica del contexto al strategy.
3. Definir v.i. y setter (constructor).
4. Dejar un método delegado en el contexto.
5. Elegir parámetros a pasar.
6. Compilar y testear.
7. Extract Parameter para permitir instanciar el strategy.
8. Replace Conditional with Polymorphism en el strategy.
9. Testear combinaciones de strategy + contexto.

//FALTA TERMINAR

EJERCICIO 4 - PEDIDOS

PREGUNTAS INVENTADAS (por la ia).

patrones

1. **Si tienes una clase cuya interfaz no es compatible con lo que tu sistema espera, pero además necesitas añadirle responsabilidades adicionales (como un log de auditoría) de forma dinámica, ¿en qué orden aplicarías los patrones Adapter y Decorator, y por qué no podrías resolver ambos problemas con uno solo de ellos?**

Primero aplicaría el patron adapter para adaptar la interfaz de la clase con la que espera el cliente (mi sistema) y luego aplicaría el patron decorator para poder añadirle responsabilidades de forma dinámica.
No es posible resolver ambos problemas con un sólo patrón ya que ambos tienen propósitos distintos. Por un lado, el adaptador se encarga de adaptar la interfaz de una clase a otra que es la que el cliente espera, pero sin agregar comportamiento adicional, para que un objeto incompatible, sea utilizable.
Por otro lado, el patron decorator no adapta la interfaz de ninguna clase y solo se encarga de añadir o quitar responsabilidades a objetos de forma dinámica en tiempo de ejecución, es decir, extiende la funcionalidad. De esta forma, no es posible utilizar sólo un Decorator si la interfaz base es incompatible, ni sólo un adapter si se requiere composición recursiva de funcionalidades. Si bien ambos patrones se encargan de "envolver" al objeto, la forma en la que lo hacen y los fines para los que lo hacen, difieren y ambos son necesarios para resolver el problema dado.

2. **Tanto el patrón State como el Strategy se basan en delegar comportamiento en un objeto ayudante mediante composición. Desde la perspectiva del acoplamiento, ¿cuál de los dos genera una dependencia más fuerte entre las clases de la jerarquía (estados o estrategias concretas) y por qué?**

Desde la perspectiva de acoplamiento, el patrón State genera mayor dependencia entre clases de la jerarquía que el patrón Strategy. Esto se debe a

que el intercambio entre los estados concretos de un objeto, es externo al mismo. De esta forma las clases de la jerarquía de estados deben conocerse entre si y saber que estado le sigue a cada uno.

3. **Al diseñar la interfaz de la clase Component en un Composite, tienes la opción de declarar los métodos de gestión de hijos (add/remove) en el nivel superior o solo en la clase Composite. ¿Cómo afecta esta decisión al cumplimiento del Principio de Segregación de la Interfaz (ISP) y qué ganancia se obtiene a cambio de violar dicho principio?**

Si se decide declarar los métodos de gestión de hijos (add/remove) en la interfaz Component, se le permite a este copomente superior mayor transparencia ya que el cliente trata a todos los objetos por igual. Sin embargo al hacer esto se viola el ISP ya que las hojas (Leaf) heredan métodos que no usan.

4. **Aunque el diagrama de objetos de un Proxy y un Decorator es casi idéntico, su relación con el ciclo de vida del objeto "real" o "servicio" es distinta. Si necesitas que el objeto pesado solo se cree cuando realmente se lo invoque por primera vez, ¿por qué es técnicamente correcto decir que el Proxy es el patrón adecuado y no el Decorator?**

En el patrón Decorator, no se gestiona el ciclo de vida (creación y destrucción) del objeto ya que el Decorator recibe un componente ya creado que debe envolver/ decorar. El patrón proxy sí gestiona el ciclo de vida (creación y destrucción) del servicio real.

5. **Analizando el patrón Template Method dentro de un framework, explica cómo este patrón garantiza que se respete el Principio de Hollywood ("No nos llame, nosotros lo llamaremos") y qué papel juegan los Hooks (métodos gancho) en este mecanismo.**

En el patrón Template Method se define un método que contiene el esqueleto del algoritmo (pasos fijos). Para permitir crear aplicaciones distintas con el mismo framework se proveen dentro del esqueleto, métodos gancho o hook methods (también conocidos como puntos de extensión) los cuales son redefinidos por los desarrolladores de aplicaciones, para crear aplicaciones a medida.

Cuando en el framework, dentro de este método plantilla se invoca a uno de estos métodos gancho redefinidos por una aplicación, es el framework quien está invocando a métodos propios de la aplicacion, logrando así la inversión de control.

6. **Según los imperativos económicos del refactoring, ¿por qué introducir un patrón como Abstract Factory al inicio de un proyecto (antes de tener múltiples familias de objetos) podría considerarse una forma negativa de deuda técnica o sobre-ingeniería?**

Introducir patrones complejos prematuramente incrementa la complejidad y el costo de mantenimiento sin un beneficio inmediato, violando el principio de ganar antes y gastar después (Net Present Value). El refactoring permite introducir el patrón solo cuando la variabilidad realmente lo requiere

7. **¿Por qué el patrón Null Object se clasifica habitualmente como un patrón de comportamiento y no como uno creacional, a pesar de que su implementación implica proveer una instancia específica que sustituye al null?**

No se suele considerar como un patron creacional ya que su propósito no tiene que ver con cómo se crea el objeto, sino con como de comunica el cliente con sus colaboradores de forma uniforme (ignorar las diferencfias entre el objeto real y el objeto nulo), encapsulando la lógicca de “no hacer nada” representando polimorficamente la ausencia de un objeto y eliminando condicionales en el flujo de ejecución.

8. dsp sigo